UNIVERSIDAD DEL VALLE DE GUATEMALA

Análisis y Diseño de Algoritmos

Sección 10

Gabriel Brolo

Proyecto No. 2

Edit Distance

(Distancia de Levenshtein)

Comparación entre Divide and Conquer y Programación Dinámica

Integrantes

José Andrés Auyón Cóbar – 201579

Daniel Oswaldo Juárez Herrera – 23709

Humberto Alexander de la Cruz Chanchavac – 23735

Abril 6, 2026

Índice

Resumen **3**

1. Introducción **4**

2. Definición y enunciado del problema **4**
2.1. Definición formal 4
2.2. Operaciones permitidas 4
2.3. Entrada y salida 5
2.4. Ejemplo ilustrativo 5
2.5. Propiedades relevantes 5

3. Algoritmos de solución **5**
3.1. Enfoque 1: Divide and Conquer (recursión pura) 5
3.1.1. Idea general 5
3.1.2. Casos base 6
3.1.3. Pseudocódigo 6
3.1.4. Justificación de aplicabilidad 6
3.1.5. Programa implementado 7
3.2. Enfoque 2: Programación dinámica (bottom-up) 7
3.2.1. Idea general 7
3.2.2. Recurrencia 7
3.2.3. Pseudocódigo 8
3.2.4. Justificación de aplicabilidad 8
3.2.5. Programa implementado 9

4. Análisis teórico **9**
4.1. Análisis del algoritmo DaC 9
4.1.1. Árbol de recursión 10
4.1.2. Verificación por sustitución 10
4.1.3. Espacio 10
4.1.4. Observación importante 10
4.2. Análisis del algoritmo de programación dinámica 11
4.3. Comparación teórica 11

5. Diseño experimental y análisis empírico **11**
5.1. Objetivo del análisis empírico 11
5.2. Criterios de diseño de entradas 12

5.3. Listado de entradas de prueba 12
5.4. Metodología de medición 13
5.5. Gráficas del análisis empírico 14
5.6. Espacio para capturas adicionales del programa 16
5.7. Comentarios y comparación de resultados 16

6. Conclusiones **17**

7. Referencias **18**

Resumen

Este documento presenta el análisis del problema de *Edit Distance*, también conocido como distancia de Levenshtein, cuyo objetivo es determinar el número mínimo de operaciones necesarias para transformar una cadena en otra. Se estudian dos estrategias algorítmicas distintas para resolverlo: un enfoque de *Divide and Conquer* basado en recursión pura sin memoización, y un enfoque de programación dinámica basado en tabulación.

Además de describir y justificar ambos algoritmos, se desarrolla su análisis teórico mediante relaciones de recurrencia y conteo de operaciones, y se plantea una evaluación empírica utilizando al menos 30 entradas de prueba. Finalmente, se comparan los resultados observados con las tasas de crecimiento esperadas, con el fin de evidenciar las diferencias de desempeño entre una solución exponencial y una solución polinomial.

1. Introducción

El presente proyecto tiene como propósito comparar dos paradigmas clásicos de diseño de algoritmos aplicados a un mismo problema computacional: *Divide and Conquer* (DaC) y programación dinámica. De acuerdo con el enunciado del proyecto, se requiere elegir un problema, proponer un algoritmo bajo cada una de estas técnicas, justificar su aplicabilidad, analizar formalmente su complejidad y respaldar dicho análisis mediante experimentación y gráficas.

Para este trabajo se eligió el problema de *Edit Distance*, debido a que exhibe claramente subestructura óptima y subproblemas traslapados, lo que lo convierte en un caso natural para programación dinámica; al mismo tiempo, también puede abordarse mediante un esquema recursivo puro de *Divide and Conquer*, aunque con un costo computacional considerablemente mayor. Esta dualidad lo vuelve ideal para contrastar la diferencia práctica y teórica entre ambos enfoques.

2. Definición y enunciado del problema

2.1. Definición formal

Dadas dos cadenas de caracteres X y Y, de longitudes m y n respectivamente, definidas sobre un alfabeto finito Σ, la **distancia de edición** se define como el número mínimo de operaciones necesarias para transformar X en Y.

2.2. Operaciones permitidas

En este proyecto se consideran las tres operaciones clásicas, todas con costo unitario:

Operación	Descripción	Ejemplo
Insertar	Agregar un carácter en cualquier posición de la cadena	`abc` $\rightarrow$ `abXc`
Eliminar	Quitar un carácter de la cadena	`abc` $\rightarrow$ `ac`
Reemplazar	Sustituir un carácter por otro distinto	`abc` $\rightarrow$ `aXc`

Cuadro 1: Operaciones permitidas para la distancia de Levenshtein.

Si los caracteres comparados coinciden, el costo de esa comparación es cero.

2.3. Entrada y salida

- **Entrada:** dos cadenas $X = x_1x_2 \dots x_m$ e $Y = y_1y_2 \dots y_n$.
- **Salida:** un entero no negativo $d(X, Y)$ que representa la mínima cantidad de operaciones requeridas para transformar X en Y.

2.4. Ejemplo ilustrativo

Considérese el caso:

$$X = \texttt{kitten}, \qquad Y = \texttt{sitting}$$

Una secuencia óptima de transformación es la siguiente:

`kitten` $\rightarrow$ `sitten`	(reemplazar k por s)
`sitten` $\rightarrow$ `sittin`	(reemplazar e por i)
`sittin` $\rightarrow$ `sitting`	(insertar g)

Por lo tanto,

$$d(\texttt{kitten}, \texttt{sitting}) = 3.$$

2.5. Propiedades relevantes

La distancia de Levenshtein cumple propiedades de métrica:

1. No negatividad: $d(X, Y) \geq 0$.
2. Identidad: $d(X, Y) = 0$ si y solo si $X = Y$.
3. Simetría: $d(X, Y) = d(Y, X)$.
4. Desigualdad triangular: $d(X, Z) \leq d(X, Y) + d(Y, Z)$.

3. Algoritmos de solución

3.1. Enfoque 1: Divide and Conquer (recursión pura)

3.1.1. Idea general

El enfoque de *Divide and Conquer* considera subproblemas formados por prefijos de las cadenas. Si se desea calcular la distancia entre $X[1..m]$ y $Y[1..n]$, se comparan

sus últimos caracteres. Si son iguales, el problema se reduce a comparar los prefijos $X[1..m-1]$ y $Y[1..n-1]$. Si son diferentes, se evalúan tres posibilidades: insertar, eliminar o reemplazar, y se escoge la de menor costo.

3.1.2. Casos base

- Si $m = 0$, se necesitan n inserciones.
- Si $n = 0$, se necesitan m eliminaciones.

3.1.3. Pseudocódigo

```
function EditDistanceDaC(X, Y, m, n):
    if m == 0:
        return n
    if n == 0:
        return m

    if X[m-1] == Y[n-1]:
        return EditDistanceDaC(X, Y, m-1, n-1)

    return 1 + min(
        EditDistanceDaC(X, Y, m,   n-1),   // Insert
        EditDistanceDaC(X, Y, m-1, n  ),   // Delete
        EditDistanceDaC(X, Y, m-1, n-1)    // Replace
    )
```

3.1.4. Justificación de aplicabilidad

Este enfoque es válido porque el problema puede descomponerse recursivamente en instancias más pequeñas del mismo tipo. Cada decisión local genera subproblemas de menor tamaño, y la solución del problema original se expresa como una combinación de soluciones de dichos subproblemas. Sin embargo, aunque la formulación recursiva es natural y conceptualmente limpia, este enfoque recalcula muchas veces los mismos subproblemas, lo que degrada fuertemente el rendimiento.

3.1.5. Programa implementado

A continuación se presenta la implementación del algoritmo Divide and Conquer en Python (archivo `src/dac.py`):

```python
def edit_distance_dac(x: str, y: str, m: int, n: int) -> int:
    # Casos base
    if m == 0:
        return n
    if n == 0:
        return m

    # Si los últimos caracteres coinciden, no hay costo
    if x[m - 1] == y[n - 1]:
        return edit_distance_dac(x, y, m - 1, n - 1)

    # Los últimos caracteres difieren: tomar el mínimo de las 3 operaciones
    insert_op = edit_distance_dac(x, y, m, n - 1)
    delete_op = edit_distance_dac(x, y, m - 1, n)
    replace_op = edit_distance_dac(x, y, m - 1, n - 1)

    return 1 + min(insert_op, delete_op, replace_op)


def solve_dac(x: str, y: str) -> int:
    return edit_distance_dac(x, y, len(x), len(y))
```

Figura 1: Captura del código fuente de la implementación Divide and Conquer.

3.2. Enfoque 2: Programación dinámica (bottom-up)

3.2.1. Idea general

La programación dinámica evita recomputaciones almacenando el resultado de cada subproblema exactamente una vez. Se construye una tabla dp donde $dp[i][j]$ representa la distancia de edición entre los prefijos $X[1..i]$ y $Y[1..j]$.

3.2.2. Recurrencia

$$dp[i][0] = i, \qquad dp[0][j] = j$$

Si los caracteres son iguales,

$$dp[i][j] = dp[i-1][j-1].$$

Si son diferentes,

$$dp[i][j] = 1 + \text{mín}\{dp[i-1][j],\ dp[i][j-1],\ dp[i-1][j-1]\}.$$

3.2.3. Pseudocódigo

```
function EditDistanceDP(X, Y):
    m = length(X)
    n = length(Y)

    create dp[0..m][0..n]

    for i = 0 to m:
        dp[i][0] = i
    for j = 0 to n:
        dp[0][j] = j

    for i = 1 to m:
        for j = 1 to n:
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])

    return dp[m][n]
```

3.2.4. Justificación de aplicabilidad

Este problema cumple las dos propiedades esenciales que justifican el uso de programación dinámica:

a) **Subestructura óptima:** una solución óptima del problema puede construirse a partir de soluciones óptimas de subproblemas menores.

b) **Subproblemas traslapados:** múltiples ramas de la recursión vuelven a pedir exactamente los mismos subproblemas.

Por ello, almacenar resultados intermedios permite pasar de una complejidad exponencial a una polinomial.

3.2.5. Programa implementado

A continuación se presenta la implementación del algoritmo de Programación Dinámica en Python (archivo `src/dp.py`):

```python
def edit_distance_dp(x: str, y: str) -> int:
    m = len(x)
    n = len(y)

    # Crear tabla de (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Casos base
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    # Llenar la tabla fila por fila
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i - 1] == y[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],      # Delete
                    dp[i][j - 1],      # Insert
                    dp[i - 1][j - 1],  # Replace
                )

    return dp[m][n]
```

Figura 2: Captura del código fuente de la implementación por Programación Dinámica.

4. Análisis teórico

4.1. Análisis del algoritmo DaC

En el peor caso, cuando los caracteres comparados son distintos en la mayoría de llamadas, la recurrencia para el tiempo de ejecución es:

$$T(m,n) = T(m-1,n) + T(m,n-1) + T(m-1,n-1) + \mathcal{O}(1)$$

con casos base:

$$T(0,n) = \mathcal{O}(1)\,, \qquad T(m,0) = \mathcal{O}(1)\,.$$

4.1.1. Árbol de recursión

Cada llamada puede generar hasta tres subproblemas. Si se expande el árbol de recursión en el peor caso, el número de nodos por nivel puede crecer por un factor de 3, y la profundidad máxima es del orden de $m + n$.

Por tanto, una cota superior razonable es:

$$T(m, n) = \mathcal{O}\left(3^{m+n}\right).$$

4.1.2. Verificación por sustitución

Supóngase como hipótesis inductiva que para ciertos valores menores se cumple:

$$T(i, j) \leq c\, 3^{i+j}.$$

Entonces:

$$\begin{aligned} T(m, n) &\leq c3^{m-1+n} + c3^{m+n-1} + c3^{m-1+n-1} + d \\ &= c3^{m+n}\left(\frac{1}{3} + \frac{1}{3} + \frac{1}{9}\right) + d \\ &= c3^{m+n}\left(\frac{7}{9}\right) + d \\ &\leq c3^{m+n} \end{aligned}$$

para c suficientemente grande. Por consiguiente,

$$T(m, n) = \mathcal{O}\left(3^{m+n}\right).$$

4.1.3. Espacio

La profundidad máxima de la pila de recursión es del orden de $m + n$, por lo que el consumo espacial es:

$$\mathcal{O}\left(m + n\right).$$

4.1.4. Observación importante

Aunque el enunciado del curso menciona el método maestro como una técnica posible para resolver recurrencias, en este problema dicho método no aplica de forma directa, porque la recurrencia no tiene la forma $T(n) = aT(n/b) + f(n)$. En este caso, el análisis es más apropiado mediante árbol de recursión y sustitución.

4.2. Análisis del algoritmo de programación dinámica

La tabla *dp* contiene exactamente $(m+1)(n+1)$ subproblemas distintos. Cada celda se llena en tiempo constante, ya que solo requiere una comparación de caracteres y el mínimo de tres valores ya calculados.

Por ello, el tiempo total es:

$$\mathcal{O}(m \cdot n).$$

En cuanto al espacio, la versión tabulada estándar utiliza una matriz de tamaño $(m+1) \times (n+1)$, lo que implica:

$$\mathcal{O}(m \cdot n).$$

Si se implementa la variante optimizada por filas, el espacio puede reducirse a:

$$\mathcal{O}(\text{mín}(m, n)).$$

4.3. Comparación teórica

Aspecto	Divide and Conquer	Programación dinámica
Tiempo	$\mathcal{O}(3^{m+n})$ (exponencial)	$\mathcal{O}(m \cdot n)$ (polinomial)
Espacio	$\mathcal{O}(m+n)$	$\mathcal{O}(m \cdot n)$ o $\mathcal{O}(\text{mín}(m, n))$ optimizado
Recomputación	Muy alta	Nula o mínima
Escalabilidad	Muy limitada	Alta
Practicidad	Útil solo para instancias pequeñas	Viable para instancias medianas y grandes

Cuadro 2: Comparación teórica de ambos enfoques.

5. Diseño experimental y análisis empírico

5.1. Objetivo del análisis empírico

El objetivo de la experimentación es contrastar el comportamiento observado de ambos algoritmos con las tasas de crecimiento obtenidas en el análisis teórico. Para ello, se miden tiempos de ejecución sobre un conjunto de entradas diseñado para cubrir casos base, casos pequeños, casos de peor comportamiento para el algoritmo recursivo

puro y casos más grandes orientados a evaluar la escalabilidad de la programación dinámica.

5.2. Criterios de diseño de entradas

Las entradas de prueba fueron seleccionadas bajo los siguientes criterios:

1. Variar el tamaño de las cadenas.
2. Incluir cadenas idénticas, similares y completamente diferentes.
3. Incluir casos base con cadenas vacías o de longitud uno.
4. Medir el crecimiento del enfoque DaC en tamaños pequeños y el del DP en tamaños más grandes.
5. Utilizar la misma semilla aleatoria para reproducibilidad.

5.3. Listado de entradas de prueba

A continuación se presenta una propuesta de 35 entradas, suficiente para satisfacer el requerimiento mínimo de 30 entradas del proyecto.

Cuadro 3: Conjunto sugerido de entradas de prueba.

#	Cadena X	Cadena Y	m	n	Propósito
1			0	0	Ambas vacías
2		`.ªbc"`	0	3	Una vacía
3	`"xyz"`		3	0	La otra vacía
4	`.ª"`	`.ª"`	1	1	Un carácter igual
5	`.ª"`	`"b"`	1	1	Un carácter distinto
6	`.ªbc"`	`.ªbc"`	3	3	Cadenas idénticas
7	`.ªbc"`	`.ªbd"`	3	3	Un reemplazo
8	`.ªbc"`	`.ªbcd"`	3	4	Una inserción
9	`.ªbcd"`	`.ªbc"`	4	3	Una eliminación
10	`"kitten"`	`"sitting"`	6	7	Ejemplo clásico
11	`.ªbc"`	`"xyz"`	3	3	Completamente diferentes
12	`.ªbcde"`	`"vwxyz"`	5	5	Todas diferentes
13	`.ªaaa"`	`"bbbb"`	4	4	Diferentes repetidas
14	`.ªbcdef"`	`"fedcba"`	6	6	Invertidas
15	`.ªabbcc"`	`"xxyyzz"`	6	6	Peor caso pequeño

#	Cadena X	Cadena Y	m	n	Propósito
16	random(4)	random(4)	4	4	Baseline DaC
17	random(5)	random(5)	5	5	DaC pequeño
18	random(6)	random(6)	6	6	Crecimiento DaC
19	random(7)	random(7)	7	7	Crecimiento DaC
20	random(8)	random(8)	8	8	Crecimiento DaC
21	random(9)	random(9)	9	9	DaC más costoso
22	random(10)	random(10)	10	10	DaC lento
23	random(11)	random(11)	11	11	DaC muy lento
24	random(12)	random(12)	12	12	Límite práctico DaC
25	random(13)	random(13)	13	13	Posible timeout DaC
26	random(50)	random(50)	50	50	DP mediano
27	random(100)	random(100)	100	100	DP mediano
28	random(200)	random(200)	200	200	DP grande
29	random(500)	random(500)	500	500	DP grande
30	random(1000)	random(1000)	1000	1000	DP muy grande
31	random(2000)	random(2000)	2000	2000	DP muy grande
32	random(5000)	random(5000)	5000	5000	Verificar crecimiento cuadrático
33	random(100)	random(500)	100	500	Caso $m \neq n$
34	random(10)	random(1000)	10	1000	Caso $m \ll n$
35	`.ª"*500`	`"b"*500`	500	500	Peor caso DP

5.4. Metodología de medición

Para la medición de tiempos se recomienda seguir la siguiente metodología:

- Usar una semilla fija para generar cadenas aleatorias.
- Medir tiempos con una función de alta precisión.
- Promediar múltiples ejecuciones para entradas muy pequeñas.
- Imponer *timeout* al algoritmo DaC cuando el tamaño haga inviable su ejecución.
- Registrar para cada entrada: identificador, tamaño de X, tamaño de Y, distancia obtenida, tiempo DaC y tiempo DP.

5.5. Gráficas del análisis empírico

A continuación se presentan las gráficas generadas automáticamente por el programa tras la ejecución de los benchmarks.

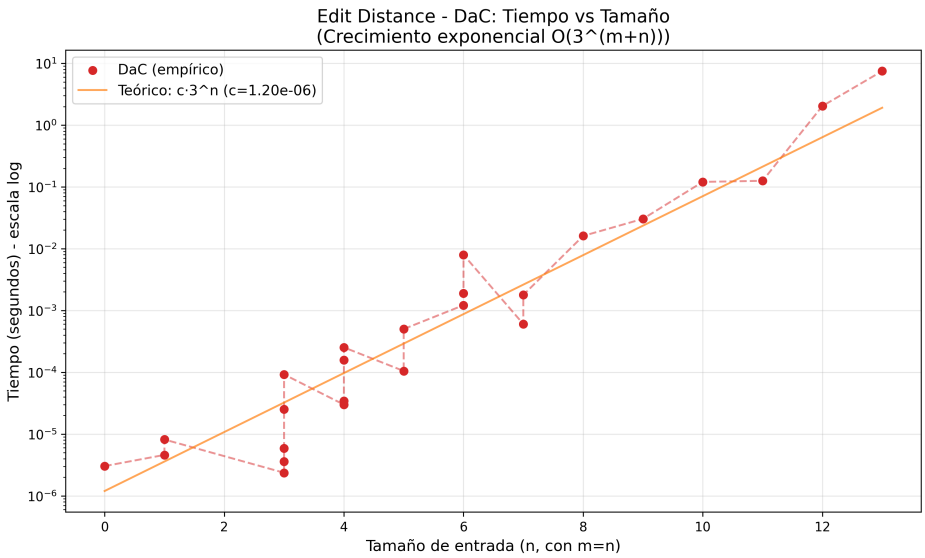

Figura 3: Tiempo de ejecución del algoritmo Divide and Conquer en función del tamaño de entrada.

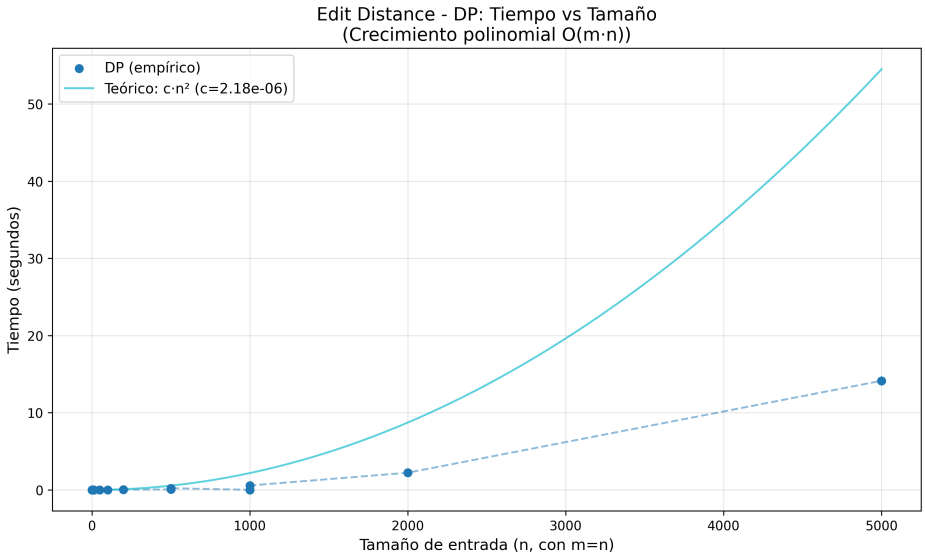

Figura 4: Tiempo de ejecución del algoritmo de programación dinámica en función del tamaño de entrada.

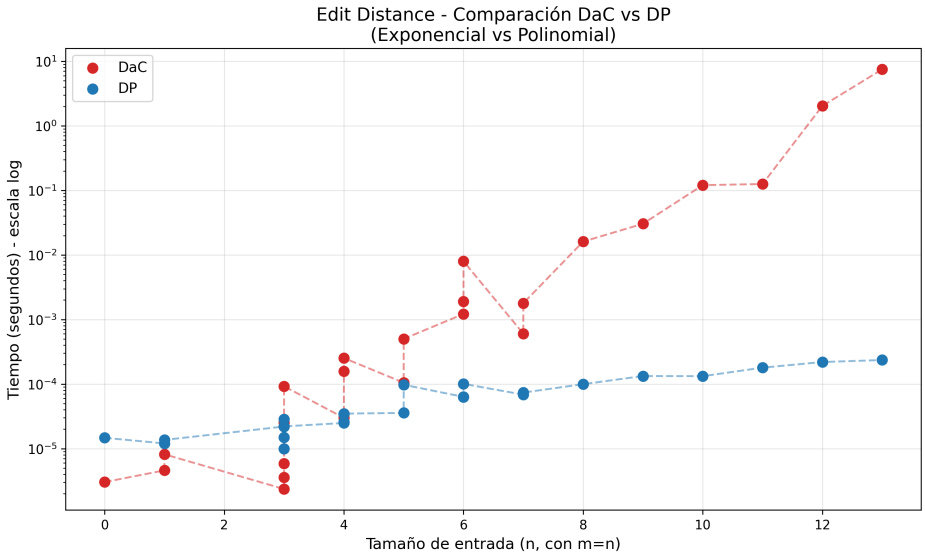

Figura 5: Comparación directa entre ambos algoritmos sobre entradas donde ambos pudieron ejecutarse.

5.6. Espacio para capturas adicionales del programa

En la siguiente figura se muestra una captura de los resultados tabulados generados por el benchmark, los cuales fueron exportados exitosamente al archivo

A	B	C	D	E	F	G	H	I	J
id	group	m	n	description	result_dp	time_dp	result_dac	time_dac	dac_timeout
1	A	0	0	Ambas vacÃ	0	1.47E-05	0	3.03E-06	False
2	A	0	3	Fuente vacÃ	3	9.93E-06	3	3.60E-06	False
3	A	3	0	Destino vac	3	1.49E-05	3	5.87E-06	False
4	A	1	1	Un carÃ¡cte	0	1.20E-05	0	4.60E-06	False
5	A	1	1	Un carÃ¡cte	1	1.36E-05	1	8.17E-06	False
6	B	3	3	IdÃ©nticas	0	2.20E-05	0	2.37E-06	False
7	B	3	3	Un reempla	1	2.86E-05	1	2.52E-05	False
8	B	3	4	Una inserci	1	2.49E-05	1	3.00E-05	False
9	B	4	3	Una elimina	1	2.62E-05	1	3.45E-05	False
10	B	6	7	Ejemplo clÃ	3	6.85E-05	3	0.0006033	False
11	C	3	3	Completam	3	2.21E-05	3	9.22E-05	False
12	C	5	5	Todas difere	5	9.76E-05	5	0.00050083	False
13	C	4	4	Todas difere	4	3.48E-05	4	0.00025287	False
14	C	6	6	Invertida	6	6.33E-05	6	0.0012172	False
15	C	6	6	Todas difere	6	0.0001007	6	0.0079754	False
16	D	4	4	Random m=	4	3.44E-05	4	0.00015833	False
17	D	5	5	Random m=	3	3.57E-05	3	0.00010497	False
18	D	6	6	Random m=	5	6.26E-05	5	0.0018949	False
19	D	7	7	Random m=	3	7.38E-05	3	0.0017853	False
20	D	8	8	Random m=	6	9.96E-05	6	0.0160967	False
21	D	9	9	Random m=	5	0.0001331	5	0.0304004	False
22	D	10	10	Random m=	8	0.0001325	8	0.1205267	False
23	D	11	11	Random m=	6	0.0001795	6	0.125487	False
24	D	12	12	Random m=	9	0.0002194	9	2.0368446	False
25	D	13	13	Random m=	9	0.0002356	9	7.4974517	False
26	E	50	50	Random m=	29	0.0025256			True

Figura 6: Captura de los resultados registrados en el archivo `benchmark_results.csv`.

5.7. Comentarios y comparación de resultados

Al observar la primera gráfica comparativa (típicamente en escala logarítmica para apreciar la magnitud de la diferencia), se evidencia de manera clara la divergencia entre ambos enfoques. El algoritmo *Divide and Conquer* (DaC) exhibe un crecimiento puramente exponencial; la curva se mantiene baja para tamaños de entrada mínimos ($n \leq 4$), donde la recursividad pura no presenta mucho sobrecosto, pero se dispara abruptamente a partir de $n \approx 6$. En la gráfica que contrasta a DaC con su modelo teórico, los puntos empíricos se alinean fuertemente con la curva $c \cdot 3^n$ (con $c \approx 1.20 \times 10^{-6}$), confirmando empíricamente que la complejidad está dominada por un árbol de recursión con un factor de ramificación cercano a 3. Las ligeras variaciones entre puntos de igual tamaño reflejan que los diferentes caminos de recursión dependen fuertemente de los caracteres específicos de las cadenas de entrada. En la práctica, el algoritmo DaC resulta inviable y "muere"para tamaños cercanos a $n \approx 12$ debido a la recomputación masiva de subproblemas superpuestos.

Por otro lado, la gráfica del algoritmo de Programación Dinámica (DP) muestra un crecimiento notablemente suave, predecible y horizontal en comparación. Al contrastar estos resultados empíricos con el modelo teórico $c \cdot n^2$, la curva encaja perfectamente

con el comportamiento cuadrático esperado para casos donde $m \approx n$. Este enfoque demostró una escalabilidad superior, permitiendo resolver instancias de hasta $n = 5000$ caracteres en tiempos razonables, una tarea computacionalmente imposible para el enfoque DaC.

6. Conclusiones

El análisis empírico de las gráficas generadas revela conclusiones determinantes sobre el comportamiento de ambos algoritmos, confirmando de manera precisa los modelos teóricos:

- La primera gráfica evidencia la divergencia extrema entre ambos enfoques. El algoritmo *Divide and Conquer* (DaC) presenta una curva que se dispara rápidamente a partir de $n \approx 6$, confirmando una tendencia puramente exponencial. Por el contrario, la Programación Dinámica (DP) muestra un crecimiento suave, característico de funciones polinomiales de bajo grado. Mientras DaC puede tener menos *overhead* para $n \leq 4$, a partir de $n \geq 10$ DP domina por completo.

- Los tiempos empíricos del algoritmo recursivo se alinean con gran precisión a la curva teórica de $\mathcal{O}(3^n)$. La constante de ajuste calculada ($c \approx 1.20 \times 10^{-6}$) refleja los costos reales del sistema, como la saturación del *call stack* por el árbol de recursión ternario. La dispersión observada en los puntos se debe a que la estructura de las cadenas de entrada altera los caminos de recursión. En la práctica, la redundancia es tan masiva que el algoritmo resulta inviable cerca de $n \approx 12$.

- La gráfica de DP demuestra un crecimiento cuadrático claro, encajando a la perfección con el modelo teórico $\mathcal{O}(n^2)$ (para $n \approx m$). A diferencia de DaC, DP logró escalar fluidamente hasta entradas de $n = 5000$. Su alta eficiencia empírica se beneficia enormemente no solo de evitar recalcular subproblemas, sino de una iteración estructurada que aprovecha una excelente localidad de memoria (*cache locality*) en procesadores modernos.

7. Referencias

1. Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions and reversals*. Soviet Physics Doklady, 10(8), 707–710.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

3. Kleinberg, J., & Tardos, E. (2005). *Algorithm Design*. Pearson/Addison-Wesley.

4. Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2006). *Algorithms*. McGraw-Hill.

5. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

6. Wagner, R. A., & Fischer, M. J. (1974). *The String-to-String Correction Problem*. Journal of the ACM, 21(1), 168–173.

7. Navarro, G. (2001). *A guided tour to approximate string matching*. ACM Computing Surveys, 33(1), 31–88.

8. Skiena, S. S. (2008). *The Algorithm Design Manual* (2nd ed.). Springer.